# NavPath Academy Technical Documentation

## 1. Architecture Overview

The current NavPath Academy application is a **frontend-only Flutter prototype**. It is designed to demonstrate the user interface, user flows, and general layout of the mobile application without relying on a functional backend.

- **Framework**: Flutter
- **Language**: Dart
- **Architecture Pattern**: Simple Stateful/Stateless widget composition with Singleton-based state management for prototyping.
- **Platform Targets**: Android, iOS, Web.

---

## 2. Current Prototype Scope

This application is deliberately scoped as a high-fidelity prototype. The primary goal is to validate UX/UI design, user journey mapping, and core visual aesthetics.

- **Data Layer**: All data is populated from local, in-memory static lists.
- **State**: State is managed in-memory via lightweight `ValueNotifier` singletons and does not persist across app restarts.
- **Connectivity**: The app does not currently make any external HTTP requests to a backend.

---

## 3. Future Development (Architecture)

To transition this prototype to a production-ready application, the following architecture is planned:

- **Backend & Database**: Integration with a scalable backend (e.g., Firebase, Supabase, or a custom Node/Python backend) to handle Relational/NoSQL database queries.
- **API Layer**: Implementing a REST or GraphQL API client (using packages like `dio` or `http`) to fetch dynamic data.
- **Authentication**: Implementing secure JWT-based or OAuth authentication.

- **State Management**: Adopting a robust state management solution (e.g., Riverpod, Provider, or BLoC) for scalable business logic.

---

# 4. Feature Documentation & Status

Each feature below is explicitly labeled with its current status:

- **[Implemented]**: Fully functional within the UI and logic constraints.
- **[Partial / Mocked]**: UI exists, but relies on mocked data or simulated logic.
- **[Planned]**: Not yet implemented in code; slated for future development.

## Authentication

- **Login/Signup UI** `[Implemented]` - Screens for email/password, Google, and OTP login are visually complete.
- **Authentication Logic** `[Mocked]` - Tapping 'Sign In' bypasses actual credential validation and routes to the Dashboard.
- **User Sessions** `[Planned]` - Secure token storage and session persistence.

## Course Browsing & Enrollment

- **Dashboard** `[Implemented]` - Displays quick stats, recommended courses, and a continue learning widget.
- **Course Catalog & Details** `[Implemented]` - Browsing categories and viewing detailed course curriculum UI.
- **Enrollment Logic** `[Mocked]` - Pressing 'Enroll' updates local in-memory state allowing access to the course.
- **Checkout & Payments** `[Mocked]` - Checkout UI exists, but payment gateway integration (e.g., Stripe/Razorpay) is simulated.

## Learning Experience

- **Mock Tests** `[Implemented]` - Fully interactive quiz UI supporting option selection and navigation.
- **Test Scoring** `[Implemented]` - Results screen automatically calculates the final score based on selected answers.
- **Video Playback** `[Partial / Mocked]` - The Video Lesson screen exists with a static placeholder for the player; actual video streaming is not yet integrated.
- **Study Material Downloads** `[Mocked]` - UI for downloading PDFs/Resources exists but is non-functional.
- **Progress Tracking** `[Planned]` - Persistent tracking of watched videos, daily streaks, and test history across devices.

### User Profile

- **Account Settings** `[Partial / Mocked]` - Settings list exists, but most options currently route to a generic Placeholder screen.
- **Edit Profile** `[Partial / Mocked]` - UI exists for updating name and email, but changes don't persist.
- **Notifications** `[Mocked]` - Static sample notifications are displayed.

---

## 5. Technology Stack

- **Core**: Flutter SDK, Dart.
- **Fonts**: `google_fonts` (Inter typeface).
- **Icons**: `cupertino_icons` , Material Icons.
- **Native Splash/Icons**: `flutter_native_splash` , `flutter_launcher_icons` .

---

## 6. Testing Documentation

*Currently, there are no automated tests. Below are the proposed test cases based on existing logic.*

### Proposed Test Cases

1. **Enrollment Logic Test**:
    - *Action*: Call `EnrollmentState.instance.enroll('imu-cet-2027')` .
    - *Expected*: `isEnrolled('imu-cet-2027')` returns `true` .
2. **Mock Test Scoring Test** (Integration):
    - *Action*: In `MockTestScreen` , select the correct index for all questions and tap Submit.
    - *Expected*: `TestResultsScreen` displays a score of 100%.

---

## 7. Self-Audit (Code vs Docs)

- **Docs reflect Code**: Yes. All documented screens, models, and state exist exactly as described.
- **Mismatches**: None found.
- **Hallucinations**: None. The document strictly identifies all backend logic, database, and APIs as "Mocked" or "Planned".